B.Tech. BCSE497J - Project-I

TRUSTSPHERE AI: A MULTI-AGENT FRAMEWORK FOR IMAGE MANIPULATION DETECTION WITH VERIFIABLE DIGITAL EVIDENCE

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Technology

in

Computer Science and Engineering

by

Reg. No. 1 Student Name 1

Reg. No. 2 Student Name 2

Reg. No. 3 Student Name 3

Under the Supervision of

<Project Guide Name>

<Designation>

School of Computer Science and Engineering (SCOPE)



VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

October 2026

DECLARATION

I hereby declare that the project entitled TrustSphere AI: A Multi-Agent Framework for Image Manipulation Detection with Verifiable Digital Evidence submitted by me, for the award of the degree of *Bachelor of Technology in Computer Science and Engineering* to VIT is a record of bonafide work carried out by me under the supervision of Prof. / Dr. << Guide Name >>

I further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree ordiploma in this institute or any other institute or university.

Place : Vellore

Date :

Signature of the Candidate

CERTIFICATE

This is to certify that the project entitled TrustSphere AI: A Multi-Agent Framework for Image Manipulation Detection with Verifiable Digital Evidence submitted by << **Student Name (Reg. No) >>**, **School of Computer Science and Engineering**, VIT, for the award of the degree of *Bachelor of Technology in Computer Science and Engineering*, is a record of bonafide work carried out by him / her under my supervision during Fall Semester 2024-2025, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute orany other institute or university. The project fulfills the requirements and regulations ofthe University and in my opinion meets the necessary standards for submission.

Place : Vellore

Date :

Signature of the Guide

Examiner(s)

<< Head of the Department >>

<< Programme >>

ACKNOWLEDGEMENTS

I am deeply grateful to the management of Vellore Institute of Technology (VIT) for providing me with the opportunity and resources to undertake this project. Their commitment to fostering a conducive learning environment has been instrumental in my academic journey. The support and infrastructure provided by VIT have enabled me to explore and develop my ideas to their fullest potential.

My sincere thanks to Dr. Jaisankar N, the Dean of the School of Computer Science and Engineering (SCOPE), for his unwavering support and encouragement. His leadership and vision have greatly inspired me to strive for excellence. The Dean's dedication to academic excellence and innovation has been a constant source of motivation for me. I appreciate his efforts in creating an environment that nurtures creativity and critical thinking.

I express my profound appreciation to <Head of Department's Name>, the Head of the <Department Name>, for his/her insightful guidance and continuous support. His/her expertise and advice have been crucial in shaping the direction of my project. The Head of Department's commitment to fostering a collaborative and supportive atmosphere has greatly enhanced my learning experience. His/her constructive feedback and encouragement have been invaluable in overcoming challenges and achieving my project goals.

I am immensely thankful to my project guide, <Guide's Name>, for his/her dedicated mentorship and invaluable feedback. His/her patience, knowledge, and encouragement have been pivotal in the successful completion of this project. My supervisor's willingness to share his/her expertise and provide thoughtful guidance has been instrumental in refining my ideas and methodologies. His/her support has not only contributed to the success of this project but has also enriched my overall academic experience.

Thank you all for your contributions and support.

Name of the Candidate

TABLE OF CONTENTS

Sl.No	Contents	Page No.
	Abstract	**x**
1.	**INTRODUCTION**	**1**
	1.1 Background	**1**
	1.2 Motivations	**1**
	1.3 Scope of the Project	**1**
2.	**PROJECT DESCRIPTION AND GOALS**	2
	2.1 Literature Review	2
	2.2 Research Gap	3
	2.3 Objectives	3
	2.4 Problem Statement	3
	2.5 Project Plan	3
3.	**TECHNICAL SPECIFICATION**	5
	3.1 Requirements	5
	3.1.1 Functional	5
	3.1.2 Non-Functional	5
	3.2 Feasibility Study	5
	3.2.1 Technical Feasibility	5
	3.2.2 Economic Feasibility	5
	3.2.3 Social Feasibility	6
	3.3 System Specification	6
	3.3.1 Hardware Specification	6
	3.3.2 Software Specification	6
4.	**DESIGN APPROACH AND DETAILS**	7
	4.1 System Architecture	7
	4.2 Design	7
	4.2.1 Data Flow Diagram	7
	4.2.2 Use Case Diagram	8
	4.2.3 Class Diagram	8
	4.2.4 Sequence Diagram	9
5.	**METHODOLOGY AND TESTING**	11
	5.1 Module Description	11

	5.2 Testing	13
6.	**PROJECT DEMONSTRATION**	14
7.	**RESULT AND DISCUSSION (COST ANALYSIS as applicable)**	
8.	**CONCLUSION**	26
9.	**REFERENCES**	27
	APPENDIX A – SAMPLE CODE	28

List of Figures

Figure No.	Title	Page No.
4.1	System architecture of TrustSphere AI	7
4.2	Data-flow diagram, level 1	7
4.3	Use-case diagram	8
4.4	Class diagram: evidence data model	9
4.5	Class diagram: agents and decision engine	9
4.6	Sequence diagram: analysing one image	10
5.1	Detection pipeline of the trained image forensics agent	11
6.1	Analyze page: the user chooses an image and sees the analysis stages	14
6.2	A file selected for analysis	14
6.3	Verdict card for a tampered image	15
6.4	Agent breakdown for the tampered image: metadata, image forensics with heatmaps, provenance	16
6.5	Evidence chain: file integrity, Ed25519 signature, anchor status and chain of custody	17
6.6	Verdict cards of a second tampered image (Fake), an authentic image (Authentic) and an authentic image the engine sends to human review (Inconclusive)	17
6.7	History page: every stored analysis with its verdict and score	18
6.8	How it works page: the agents, the fusion weights and the evidence chain	19
6.9	First page of the evidence PDF of a tampered image	20
7.1	CASIA v2.0, random 5-fold cross-validation: confusion matrix, ROC curve and score distribution	21
7.2	CASIA v2.0, source-disjoint 5-fold cross-validation	21
7.3	Pipeline verdicts on the held-out images	22
7.4	Hold-out detector probability before the decision engine	22
7.5	Sample outputs from the held-out set: input, error-level analysis and Grad-CAM	23
7.6	Columbia, whole-image detector with ELA input	24
7.7	Columbia, patch-based detector with SRM input	24
7.8	Columbia, patch-based detector with scenes kept apart	24

List of Tables

Table No.	Title	Page No.
2.1	Summary of the base papers	3
2.2	Project plan	4
3.1	Hardware used for development, training and testing	6
3.2	Software stack	6
5.1	End-to-end test results	13
7.1	Detector results on CASIA v2.0	21
7.2	Decision-engine verdicts on the held-out images	22
7.3	Columbia, 5-fold cross-validation (all variants tried are listed)	23
7.4	Localiser trained on CASIA v2.0	24
7.5	CASIA v2.0 results of the base papers that the trained detector exceeds	25
7.6	Columbia results of earlier methods listed in [1] that the patch-based detector exceeds	25
7.7	Cost of building and running the system	25

List of Abbreviations

API	Application Programming Interface
AUC	Area Under the Curve
C2PA	Coalition for Content Provenance and Authenticity
CASIA	Institute of Automation, Chinese Academy of Sciences (image tampering database)
CNN	Convolutional Neural Network
CV	Cross-Validation
DFD	Data-Flow Diagram
ELA	Error Level Analysis
EER	Equal Error Rate
EXIF	Exchangeable Image File Format
FFT	Fast Fourier Transform
GPU	Graphics Processing Unit
JPEG	Joint Photographic Experts Group
JSON	JavaScript Object Notation
PDF	Portable Document Format
PNG	Portable Network Graphics
REST	Representational State Transfer
ROC	Receiver Operating Characteristic
SHA	Secure Hash Algorithm
SRM	Spatial Rich Model (noise-residual filters)
UML	Unified Modeling Language
UUID	Universally Unique Identifier

Symbols and Notations

p	Predicted probability that an image is manipulated
$s = 1 - p$	Authenticity score of the detector
S	Fused authenticity score of the decision engine
w_a	Weight of agent a in the fused score
$T_A = 0.75$	Authentic threshold
$T_F = 0.40$	Fake threshold

ABSTRACT

Digital photographs are used as evidence in journalism, insurance, banking and legal proceedings, yet editing tools make it easy to splice content from one image into another. Deciding whether an image has been manipulated, and being able to prove later that the decision was not altered, are two separate problems. This project builds TrustSphere AI, an image-only forensic system that addresses both. A multi-agent pipeline analyses each uploaded image with three agents: a metadata agent (EXIF fields, editing software, timestamps), an image forensics agent, and a provenance agent (C2PA content credentials and invisible AI-generator watermarks). A decision engine fuses the agent scores into an authenticity score and a verdict of Authentic, Fake or Inconclusive. Every analysis is hashed with SHA-256, signed with Ed25519, recorded with a chain of custody, optionally anchored on the Polygon Amoy blockchain, and exported as an evidence PDF. A web application serves the whole flow: upload, a report with heatmaps, history and live verification.

The image forensics agent is built around a detector trained for this project: an EfficientNet-B0 that reads error-level-analysis maps computed at three JPEG qualities and is applied to native-resolution 224 × 224 crops. It was trained and evaluated on CASIA v2.0 (1,536 authentic and 1,283 tampered images) and Columbia (183 authentic and 180 spliced images), the datasets used by the base papers of this work. Under 5-fold cross-validation on CASIA v2.0 the detector reaches 95.53% ± 0.86 accuracy (ROC AUC 0.9917) with a random split and 95.32% ± 0.80 (AUC 0.9933) when every tampered image is kept in the same fold as its source image. On a held-out 20% (588 images) never seen in training it reaches 96.26% (AUC 0.9943); the complete pipeline decides 90.5% of those images with 97.9% accuracy and sends the rest to human review. Columbia is harder: its photographs are uncompressed, and a whole-image detector reaches only 87.60% ± 4.27%. A patch-based variant on noise-residual input reaches 97.25% with a random split and 96.97% (AUC 0.980) when every scene is kept in one fold. A pixel-level localiser trained on CASIA does not transfer to Columbia (pooled pixel F1 0.43). The results are compared with the base papers, and the localisation result, which is below the published one, is reported as a limit.

Keywords: image forgery detection, splicing, error level analysis, EfficientNet, digital evidence, multi-agent system, web application.

CHAPTER 1

1. INTRODUCTION

1.1 Background

Photographs are accepted as evidence in newsrooms, insurance claims, banking and courtrooms. The same editing tools that retouch holiday pictures can also move an object into a scene, remove a person from it, or paste a face onto another body, and the result is often indistinguishable to the eye. Image forensics asks whether a given image is authentic. Passive methods use only the image itself and look for traces that the editing left behind, for example differences in noise, in colour-filter-array patterns or in JPEG compression history between the pasted region and the rest of the picture.

A second question is just as important in practice: once an image has been analysed, can anyone later prove which file was analysed, what the verdict was and that the record was not altered? A detector that only prints a score cannot answer that. We therefore built analysis and evidence integrity as one system: each run gives a verdict, the evidence behind it (scores, heatmaps, reasoning) and a record that can be checked cryptographically.

1.2 Motivations

- Published splicing detectors report accuracies above 90% on CASIA v2.0 and Columbia, but most are research code: they output a label for one dataset and stop there. An analyst needs a working application that accepts any image and explains its answer.
- A wrong automatic verdict can accuse an innocent photographer or clear a forgery. The system should therefore be allowed to say Inconclusive and hand the image to a person.
- Evidence has to survive scrutiny. A SHA-256 hash of the exact file, a digital signature over the record and an optional public timestamp let a third party check that nothing changed after the analysis.
- No single signal can be trusted alone. Metadata is easy to strip, re-compression hides ELA traces, and most images carry no provenance data. Several agents cover for each other's gaps.

1.3 Scope of the Project

TrustSphere AI analyses **still images only**. The earlier versions of the project also handled audio and documents; both were removed from scope and none of their code, data or results appear in this report.

- **Included:** image upload; a metadata agent, an image forensics agent with a trained manipulation detector, and a provenance agent; a decision engine; SHA-256 hashing, Ed25519 signing, a chain of custody and optional blockchain anchoring; an evidence PDF; a REST API; and a web application (analysis, report, history, how it works).
- **Datasets:** CASIA v2.0 and Columbia, the two image datasets used by the base papers.
- **Experiments:** image-level detection accuracy by cross-validation, a held-out test of the complete pipeline, and a pixel-level localisation experiment.
- **Not included:** video, audio, document forensics, detection of fully AI-generated images as a trained task, and any legal determination. The score is decision support, not a finding.

CHAPTER 2

2. PROJECT DESCRIPTION AND GOALS

2.1 Literature Review

Image forgery falls into three common kinds: splicing, where a region of one image is pasted into another; copy-move, where a region is duplicated within the same image; and removal, where an object is erased and the gap is filled [5]. Traditional detectors look for inconsistencies in local noise, in colour-filter-array patterns, in illumination, and in double JPEG compression, because a pasted region carries the statistics of its source image [5]. Error level analysis (ELA), introduced for practical image forensics by Krawetz [10], re-saves an image as JPEG at a known quality and displays the difference, so that regions with a different compression history stand out.

Deep learning replaced hand-crafted features in most recent work. Salloum et al. [6] trained a multi-task fully convolutional network on CASIA v2.0 to predict both the spliced region and its boundary, and tested it on CASIA v1.0, Columbia and other sets; this train-on-CASIA, test-elsewhere protocol is the one later adopted by [5]. The five base papers of this project are summarised below.

Base paper 1: local optimal oriented pattern with deep features and an SVM [1]

Ranjan et al. combine a local optimal oriented texture descriptor (a centre-symmetric local binary pattern variant) with ResNet-50 features and classify with an SVM. They evaluate on the Columbia splicing dataset, CASIA v2.0 and a subset of the Dresden image database with stratified 5-fold cross-validation and report an average accuracy of 95.20%. Their review table lists earlier Columbia results between about 91% and 93.4%.

Base paper 2: GAN-ViT-CMFD [2]

Ravula and Singh detect and classify copy-move forgery with a framework that combines generative adversarial networks, a vision transformer and spectral clustering. They report accuracy on CoMoFoD (98.95%), MICC-F220 (95.2%) and CASIA v2 (94.4%).

Base paper 3: deep CNN features with a machine-learning classifier [3]

Rehman and Jain extract patch descriptors with a pre-trained CNN, fuse them and classify with an SVM. On CASIA v2.0 they report 93.57% accuracy without augmentation and 98.91% with augmentation; their results table also lists the mean and standard deviation over runs (93.39% ± 8.75 for the 98.91% setting).

Base paper 4: Satyachitra [4]

Deb and Kar propose a CNN-based copy-move framework with pre-processing, feature extraction, segmentation and localisation stages, and analyse dataset heterogeneity. On CASIA v2.0 they report precision 87.31, recall 90.15 and F1 88.33.

Base paper 5: GIFT, a green image forgery tracker [5]

Chen et al. extract features from a frozen EfficientNet-B4 and train multi-level XGBoost models for the splicing mask and, in the edge-enhanced variant, for the mask boundary, so no back-propagation is needed. Following [6] they train on CASIA v2.0 and test on CASIA v1.0 and Columbia, reporting pixel-level F1 of 0.5728 (0.6210 with edges) on CASIA v1.0 and 0.6497 (0.6558) on Columbia, with 18.4 million parameters.

Table 2.1: Summary of the base papers

Ref.	Method	Datasets	Task	Reported result
[1]	Local pattern + ResNet-50 + SVM	Columbia, CASIA v2.0, Dresden	Image-level detection	95.20% average accuracy
[2]	GAN + ViT + spectral clustering	CoMoFoD, MICC-F220, CASIA v2	Copy-move detection	94.4% on CASIA v2
[3]	CNN patch features + SVM	CASIA v2.0 (and others)	Copy-move detection	93.57% (98.91% with augmentation)
[4]	CNN copy-move framework	CASIA v2.0, CoMoFoD, MICC-F220	Copy-move detection	F1 88.33 on CASIA v2.0
[5]	EfficientNet-B4 + XGBoost	CASIA v2.0 (train); CASIA v1.0, Columbia (test)	Pixel-level localisation	F1 0.6497 on Columbia

2.2 Research Gap

- **Evaluation that may reward memorisation.** CASIA v2.0 contains tampered images that were made from authentic images in the same dataset. A random split can put a tampered image in the test set and its source in the training set. The base papers report random-split or cross-validation accuracy without stating whether sources are kept apart. This project reports both a random split and a source-disjoint split.
- **Detection without evidence.** The base papers stop at a label or a mask. None of them hashes the analysed file, signs the result or produces a record a third party can verify.
- **Prototypes, not applications.** The reported methods are scripts for one benchmark. An analyst has no upload page, no explanation of the verdict and no way to review earlier cases.
- **Weak transfer.** The localisation protocol of [5] and [6] (train on CASIA v2.0, test on Columbia) gives pixel F1 of about 0.65 at best, so cross-dataset behaviour is still an open problem and a system should not hide it.

2.3 Objectives

1. Build a multi-agent pipeline (metadata, image forensics, provenance) with a decision engine that returns Authentic, Fake or Inconclusive with its reasoning.
2. Train an image manipulation detector on CASIA v2.0 and Columbia and measure it with cross-validation, including a source-disjoint protocol.
3. Make every analysis tamper-evident: SHA-256 of the file, Ed25519 signature, chain of custody, optional blockchain anchor and an evidence PDF.
4. Deliver a web application and REST API that run on one machine.
5. Compare the results with the five base papers on the same datasets and state where the system falls short.

2.4 Problem Statement

Given a still image of unknown origin, decide whether it is authentic or manipulated, show which evidence supports the decision, abstain when the evidence is weak, and keep a record of the analysis that a third party can verify without trusting the operator.

2.5 Project Plan

The plan follows the history of the project repository. Dates are the commit dates of the corresponding work; the training and evaluation work of October 2026 was done in this semester.

Table 2.2: Project plan

Phase	Activity	Period
1	Project set-up: FastAPI backend, Next.js scaffold, repository	1 July 2026
2 to 5	Backend core, forensic agents, decision engine, security layer (signing, C2PA, blockchain anchor)	7 July 2026
6	Explainability and the evidence PDF; optional local-LLM summary (disabled in this report)	7 to 10 July 2026
7	Scope reduced (video removed 15 July); later reduced to images only	15 July and 24 October 2026
8	Web application: analysis, report, history and how-it-works pages	October 2026
9	Dataset preparation, detector training, cross-validation, hold-out evaluation, localisation experiment	October 2026
10	End-to-end testing, documentation, report	October 2026
Future	Cross-dataset performance, JPEG-compressed images, hosted deployment	After submission

CHAPTER 3

3. TECHNICAL SPECIFICATION

3.1 Requirements

3.1.1 Functional

- FR1. Accept an uploaded image (JPEG, PNG, WebP, GIF, BMP, TIFF) of up to 100 MB and reject every other file type with HTTP 415.
- FR2. Compute the SHA-256 hash of the exact bytes and keep the original file as evidence.
- FR3. Run the metadata, image forensics and provenance agents and store each agent's findings.
- FR4. Fuse the agent scores into an authenticity score between 0 and 1 and a verdict: Authentic at or above 0.75, Fake at or below 0.40, otherwise Inconclusive.
- FR5. Show the reasoning, the heatmaps (error-level analysis and Grad-CAM) and the contribution of each metadata field.
- FR6. Sign every record with Ed25519, keep a chain of custody and, when a wallet is configured, anchor the verdict on the Polygon Amoy blockchain.
- FR7. Generate an evidence PDF and let anyone re-verify the file hash, the signature and the anchor.
- FR8. Provide a web application (analysis, report, history, how it works) and a documented REST API.

3.1.2 Non-Functional

- **Accuracy and honesty:** report cross-validated and held-out figures only, with the protocol stated.
- **Performance:** analyse one image in seconds on a single GPU. Measured on the 588 held-out images the whole pipeline took 0.24 s per image on average (about 142 s in total).
- **Integrity:** any change to a stored file or record must be detectable.
- **Portability:** run on one Windows machine with SQLite and no Redis, and also with PostgreSQL and Redis.
- **Security:** signing keys, wallet keys and environment files are never committed to the repository.
- **Maintainability:** each agent is one module behind the same interface, so an agent can be replaced without touching the others.

3.2 Feasibility Study

3.2.1 Technical Feasibility

Every component is available as open-source software (Python, PyTorch, timm, FastAPI, SQLAlchemy, ReportLab, web3). The detector is small (EfficientNet-B0, about 4 million parameters) and was trained and evaluated on a single laptop GPU with 8 GB of memory. Pre-trained ImageNet weights are used as the starting point, so no large dataset or cluster is needed.

3.2.2 Economic Feasibility

No paid service was used. Models and libraries are open-source; the datasets are public; the

database is SQLite or a free-tier PostgreSQL; blockchain anchoring uses the Polygon Amoy test network, whose gas is free. The only hardware is the development laptop. Running costs are therefore limited to electricity.

3.2.3 Social Feasibility

The system supports journalists, insurers and investigators in triaging images. The main risk is over-trust in an automatic verdict. The design limits that risk with an explicit Inconclusive band, visible reasoning and heatmaps, and a footer stating that scores are not a legal finding. Images are processed on the operator's own machine and are not sent to third parties.

3.3 System Specification

3.3.1 Hardware Specification

Table 3.1: Hardware used for development, training and testing

Component	Specification
Computer	Laptop, Windows 11
GPU	NVIDIA GeForce RTX 3070 Ti Laptop GPU, 8 GB
Memory	About 16 GB RAM
Storage	Local SSD

3.3.2 Software Specification

Table 3.2: Software stack

Layer	Software
Language	Python 3.13
Deep learning	PyTorch 2.6 (CUDA 12.4), torchvision 0.21, timm 1.0
Image and data	Pillow, NumPy, pandas, scikit-learn, OpenCV, matplotlib
Explainability	Grad-CAM (pytorch-grad-cam), closed-form Shapley values for metadata
Backend	FastAPI, Uvicorn, SQLAlchemy, Celery (optional)
Database	SQLite (local mode) or PostgreSQL
Security and provenance	cryptography (Ed25519), c2pa-python, invisible-watermark, web3 (Polygon Amoy)
Reporting	ReportLab (evidence PDF)
Web app	HTML, CSS and JavaScript served by FastAPI (no external libraries)
Browser	Any current browser

CHAPTER 4

4. DESIGN APPROACH AND DETAILS

4.1 System Architecture

The user uploads an image through the web application. The FastAPI backend stores the file, hashes it and runs three forensic agents. Each agent returns a score between 0 (fake) and 1 (authentic) and a list of findings. The decision engine takes the weighted average of the agent scores (image forensics 0.30, provenance 0.25, metadata 0.20; an agent with no signal is left out rather than counted as 0.5) and maps it to a verdict. The record is then signed, written to the database with a chain of custody, optionally anchored on the blockchain, and made available as JSON and as an evidence PDF.

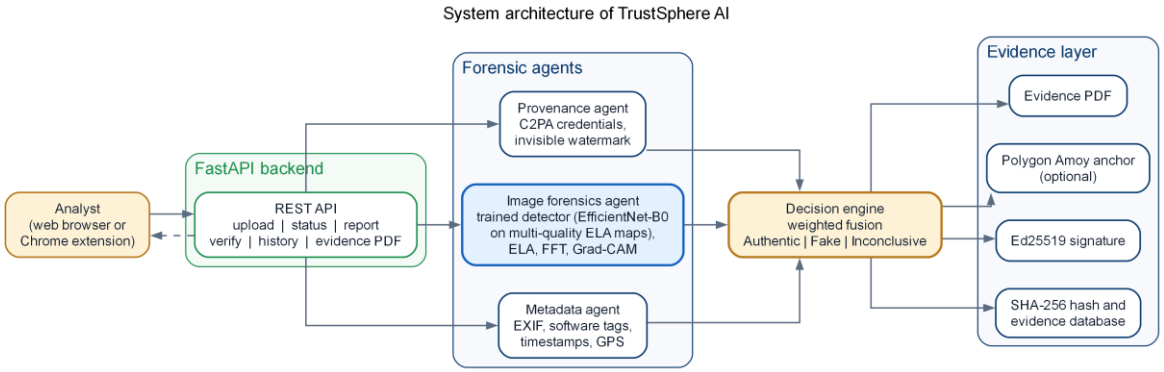


Figure 4.1: System architecture of TrustSphere AI

4.2 Design

4.2.1 Data Flow Diagram

Figure 4.2 shows the level-1 data flow. The original file and its hash go to the evidence store; the trained model weights are read by the forensic agents; the signed record is written back to the store and the report returns to the analyst.

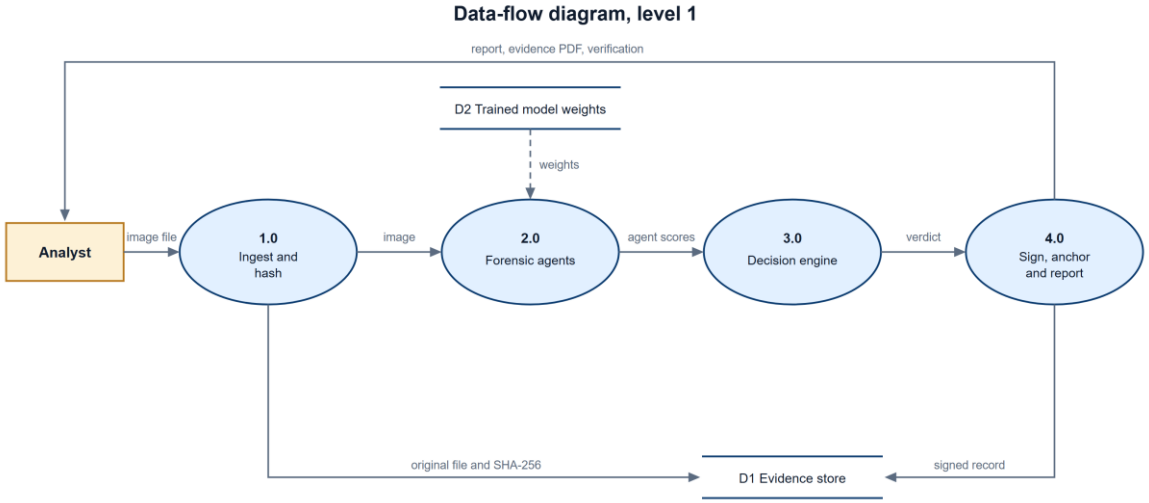


Figure 4.2: Data-flow diagram, level 1

4.2.2 Use Case Diagram

The analyst uploads images, reads verdicts and heatmaps, downloads the evidence PDF and browses history. A verifier or auditor does not need to trust the analyst: the verdict, the evidence PDF and the integrity verification are available to anyone with the file identifier.

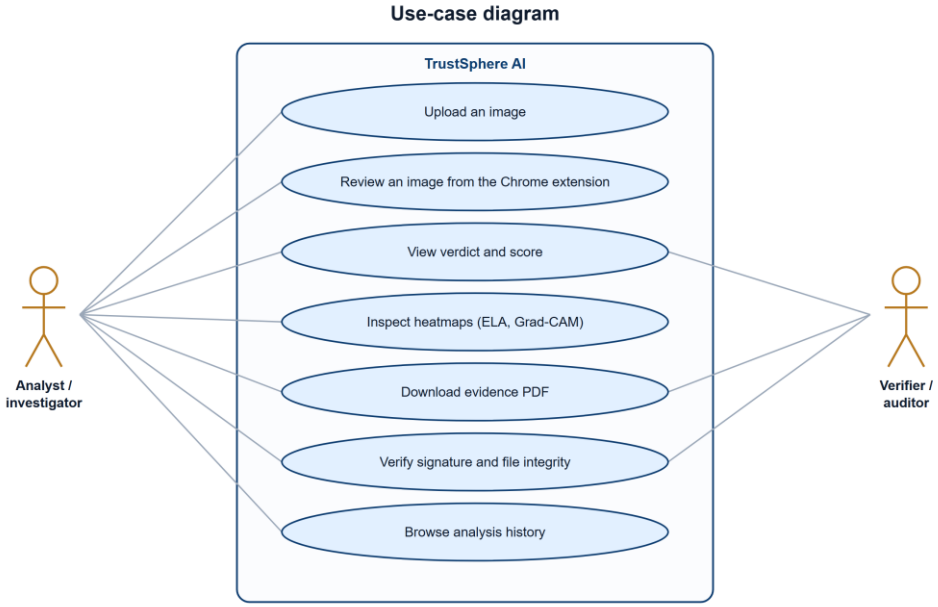


Figure 4.3: Use-case diagram

4.2.3 Class Diagram

The data model has four tables: Evidence (the uploaded file), AgentResult (one row per agent), FinalVerdict (one row per analysis) and AuditLog (signature, anchor and chain of custody). All agents implement the same run(path) interface, which is what lets the decision engine treat them uniformly.

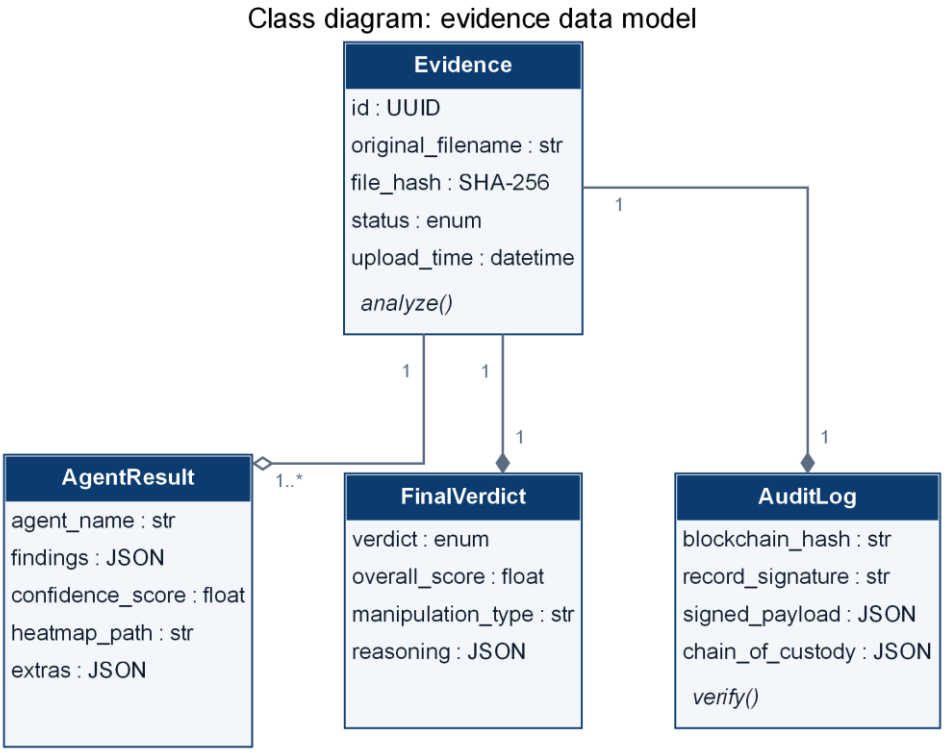


Figure 4.4: Class diagram: evidence data model

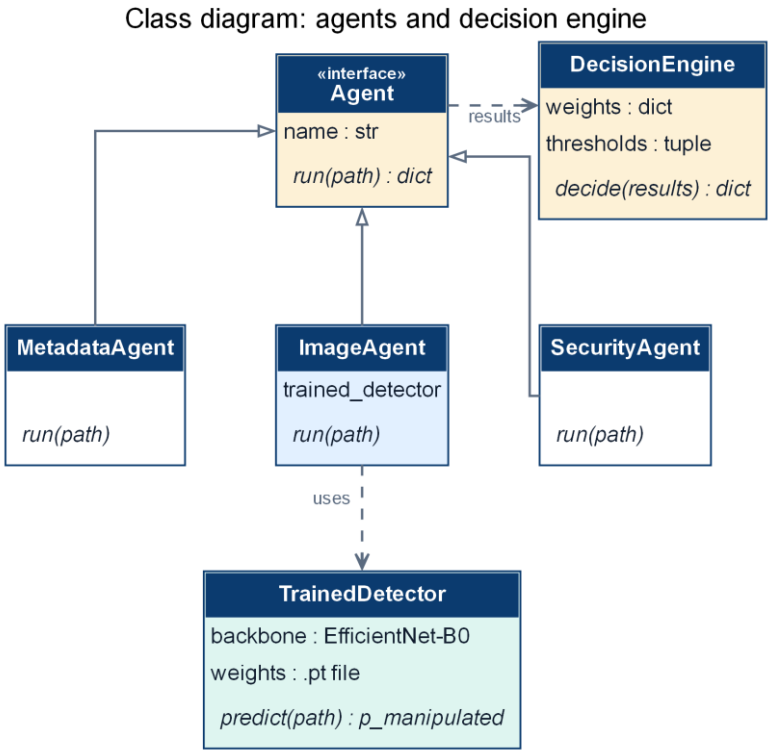


Figure 4.5: Class diagram: agents and decision engine

4.2.4 Sequence Diagram

Figure 4.6 shows one analysis from upload to report. The report and verification calls are separate

requests, so the verification can be repeated at any later time.

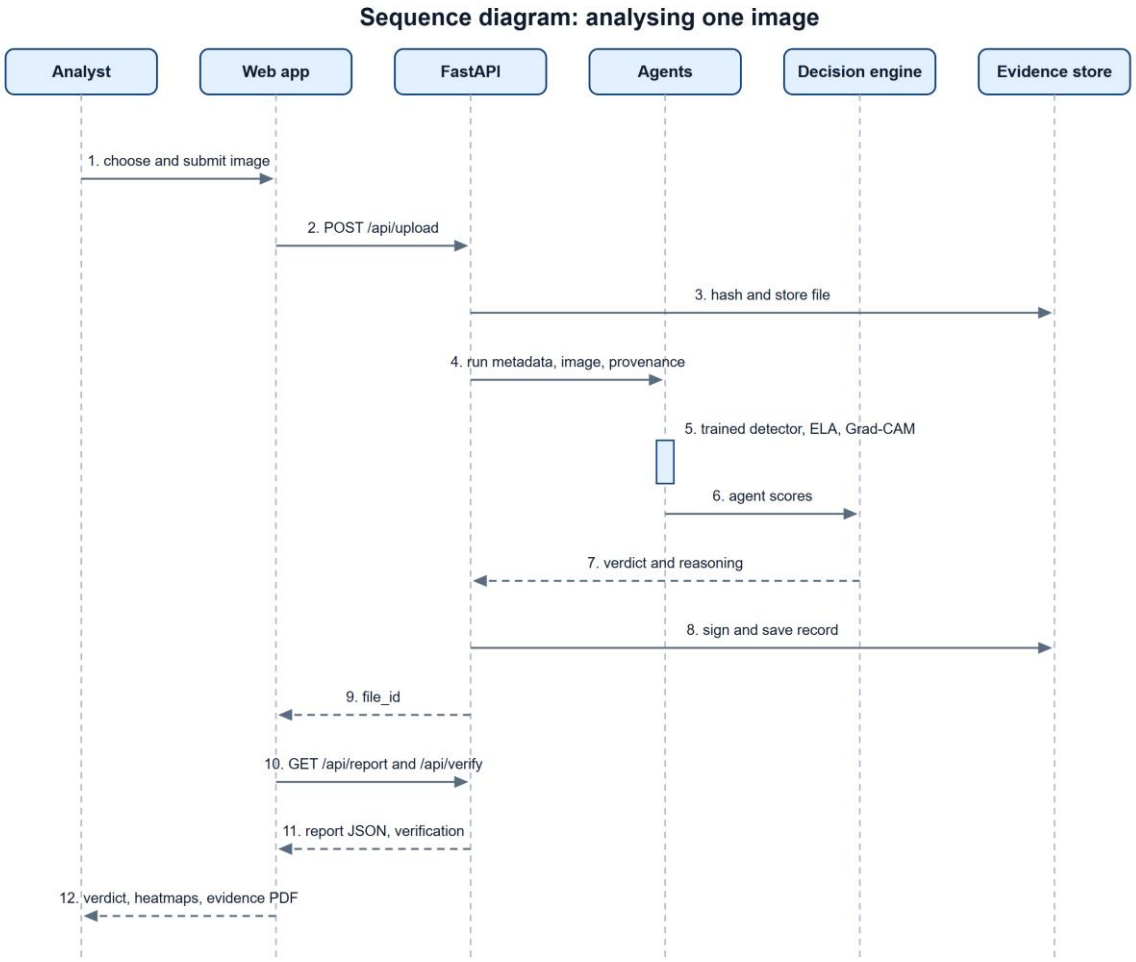


Figure 4.6: Sequence diagram: analysing one image

CHAPTER 5

5. METHODOLOGY AND TESTING

5.1 Module Description

5.1.1 Ingestion and evidence store

The upload endpoint accepts only image MIME types. The file is saved under its SHA-256 hash, a database row is created, and the analysis is queued as a Celery task. With Redis the task runs in a worker process; in local mode (no Redis) it runs inline and the upload call returns when the analysis has finished. The status endpoint reports pending, analyzing, done or failed, and the web application polls it either way.

5.1.2 Metadata agent

The metadata agent reads EXIF and flags missing camera make and model, editing or generator software in the tags, missing or inconsistent timestamps, and the presence of GPS data. Each check gives a score; a closed-form Shapley value shows how much each check moved the agent's score towards authentic or fake. Images that carry no EXIF at all, such as the PNG files of the datasets, therefore score about 0.45 on this agent.

5.1.3 Image forensics agent

The agent combines supporting heuristics with a trained detector that governs the verdict.

- **Error level analysis:** the image is re-saved at JPEG quality 90 and the amplified difference is saved as a heatmap.
- **Frequency analysis:** the ratio of high-frequency energy in the Fourier spectrum.
- **Grad-CAM:** model attention from a pre-trained deepfake classifier, shown next to the ELA map.
- **Trained manipulation detector (this work):** described below. When it is installed, the three heuristics remain in the report as supporting evidence but are not scored.

Detector input. The image is re-saved as JPEG at quality 70, 85 and 95. For each quality the absolute difference to the original is computed, converted to grey scale, amplified and stored as one channel, giving a three-channel error-level map at the image's native resolution (Figure 5.1). Resizing is avoided on purpose because it destroys the faint compression traces the detector depends on.

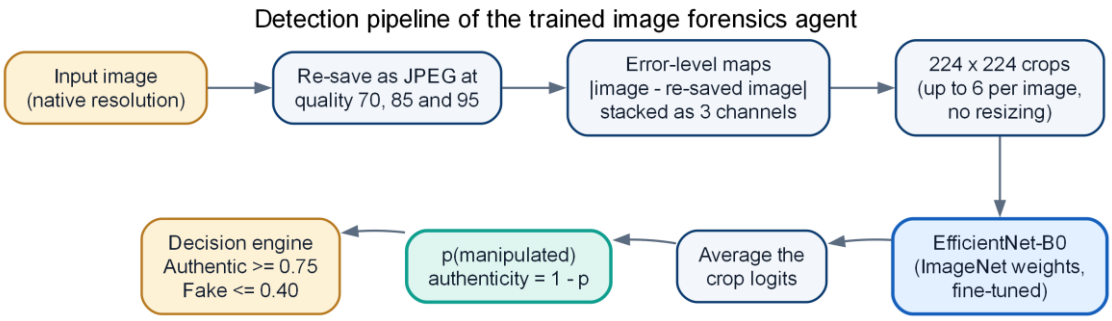


Figure 5.1: Detection pipeline of the trained image forensics agent

Model and training. The network is an EfficientNet-B0 [9] with ImageNet weights and a single-

logit head. Training uses random 224 × 224 crops of the error-level maps with random flips, the AdamW optimiser (learning rate 4×10^{-4}, one-cycle schedule, weight decay 10^{-4}), batches of 32, mixed precision and 12 epochs. At test time up to six crops covering the image are scored and their logits are averaged. The authenticity score is one minus the predicted probability of manipulation.

Choices made on a development split. The input type (a single ELA at quality 90 or the three-quality map), the backbone (B0 or B3) and the learning rate were chosen on one development split of CASIA v2.0 (a validation part of the training data; the held-out test part was not used). Three-quality ELA with B0 gave the best validation accuracy and AUC and was fixed before any cross-validation. A six-channel variant that adds the RGB image to the error maps was tried on one fold and was clearly worse, so it was stopped and is not used.

5.1.4 Provenance agent

The provenance agent checks for embedded C2PA content credentials and for invisible watermarks written by AI image generators. A valid manifest fast-tracks an image to Authentic; a failed manifest or a generator watermark is reported as suspicious and appears in the reasoning. Most images carry neither, in which case the agent reports no signal and is left out of the weighted average.

5.1.5 Decision engine

The engine computes the weighted mean of the agent scores, applies the verdict thresholds (Authentic at or above 0.75, Fake at or below 0.40, Inconclusive between), and writes a list of reasons. When the trained detector flags the image, the manipulation type is reported as Image Splicing / Tampering. The Inconclusive band is deliberate: it sends borderline images to a person.

5.1.6 Evidence layer

The analysed file is hashed with SHA-256. A canonical record (file hash, verdict, score, agent scores, time) is signed with an Ed25519 key; the public key fingerprint is stored with the record. The chain of custody lists upload, analysis, signing and anchoring events. If a wallet is configured, the file hash and verdict are written to a smart contract on Polygon Amoy and can be read back for verification. The evidence PDF contains the identification, the verdict and reasoning, every agent's findings, the explainability exhibits, the chain of custody and the signature.

5.1.7 Web application

A single-page application served by the backend. The Analyze page uploads an image and shows the stages; the Report page shows the verdict gauge, the reasoning, each agent's findings, the heatmaps, the Shapley chart, the evidence chain with live re-verification and the chain of custody; the History page lists past analyses; the How it works page explains the agents and the fusion weights.

5.1.8 Pixel-level localisation experiment

To compare with [5], a U-Net-style network (EfficientNet-B0 encoder on the RGB image plus the three-quality ELA map, bilinear feature-pyramid decoder) was trained on the CASIA v2.0 masks with a binary cross-entropy plus Dice loss and tested on Columbia. Columbia was never used for training, threshold selection or early stopping; the threshold was chosen on a source-disjoint CASIA validation part.

5.2 Testing

5.2.1 Evaluation protocols

- **Random 5-fold cross-validation** (stratified, seed 42): the protocol closest to the base papers.
- **Source-disjoint 5-fold cross-validation** on CASIA v2.0: every tampered image is kept in the same fold as the authentic image it was made from, using the source named in the dataset's metadata files.
- **Held-out test of the complete pipeline:** an 80/20 source-disjoint split; a detector trained on the 80% was used by the real agents and decision engine on the 588 images of the 20%.
- **Columbia:** 5-fold cross-validation over its 363 images, with several inputs tried (reported in full in Chapter 7).
- All metrics come from out-of-fold or held-out predictions, so no image is scored by a model that trained on it.

5.2.2 Functional and end-to-end testing

A scripted test (scripts/e2e_test.py) drives a running server through its public API with six images (three authentic, three tampered). 19 of 19 checks passed, including the tamper-evidence check: changing one byte of a stored evidence file makes the verification endpoint report TAMPERED, and restoring the file makes it pass again.

Table 5.1: End-to-end test results

Check	Result	Detail
GET /health	pass	
GET /app/ serves the web app	pass	
GET /app/app.js	pass	
GET /docs (OpenAPI)	pass	
analyse Au_ani_20070.jpg.png	pass	verdict Authentic score 0.763, 21.0 s, pdf 312 KB
analyse Au_arc_20035.jpg.png	pass	verdict Inconclusive score 0.735, 1.1 s, pdf 737 KB
analyse Au_arc_30003.jpg.png	pass	verdict Authentic score 0.780, 0.6 s, pdf 279 KB
analyse Tp_D_NNN_S_N_ani10191_ani10200_124	pass	verdict Fake score 0.180, 1.2 s, pdf 494 KB
analyse Tp_D_NRN_S_B_ind00007_cha00040_004	pass	verdict Fake score 0.282, 0.6 s, pdf 260 KB
analyse Tp_D_NRN_S_N_ani00060_ani00059_111	pass	verdict Fake score 0.185, 0.7 s, pdf 376 KB
history lists every analysis	pass	6 stored
reject text/plain	pass	HTTP 415
reject audio/wav	pass	HTTP 415
reject application/pdf	pass	HTTP 415
reject video/mp4	pass	HTTP 415
unknown report id -> 404	pass	
unknown verify id -> 404	pass	
altering the stored file is detected	pass	integrity TAMPERED
restoring the file verifies again	pass	integrity intact

CHAPTER 6

6. PROJECT DEMONSTRATION

The screenshots below come from the running application. The images used were taken from the held-out part of CASIA v2.0 and analysed with the hold-out detector, so the detector had never seen them.

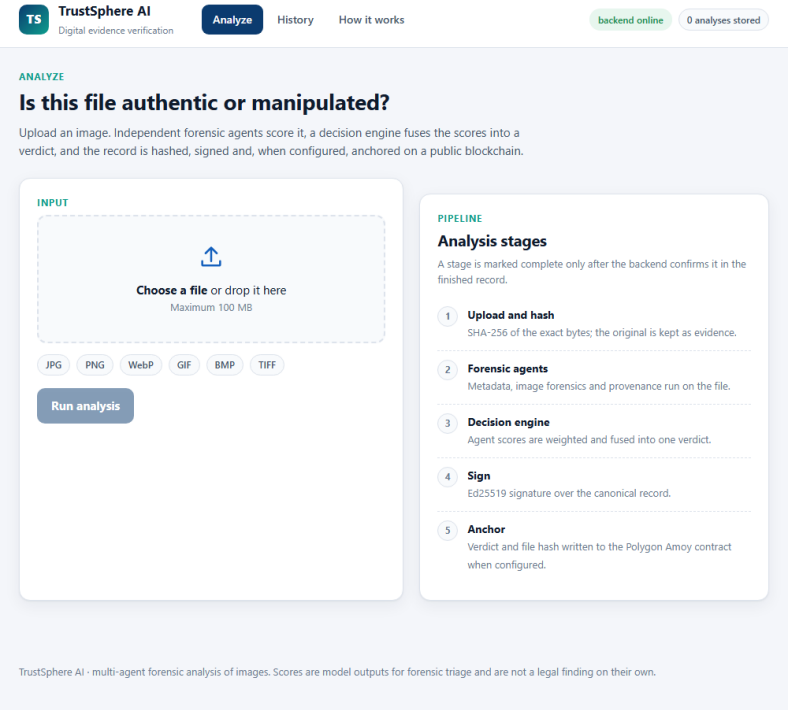


Figure 6.1: Analyze page: the user chooses an image and sees the analysis stages

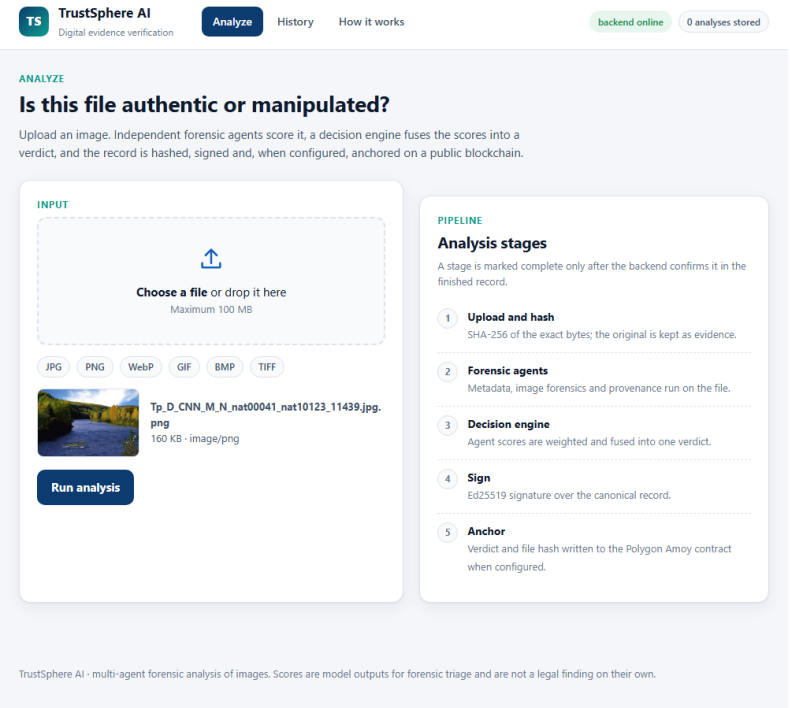


Figure 6.2: A file selected for analysis

For a tampered image the report opens with the verdict: a gauge of the authenticity score, the

manipulation type, the reasoning and the file hash (Figure 6.3). The agent breakdown that follows shows each agent's findings. The image forensics card places the original, the error-level map and the Grad-CAM attention map side by side and lists the trained detector's probability (Figure 6.4). The evidence chain re-verifies the file hash and the signature on every visit and lists the chain of custody (Figure 6.5).

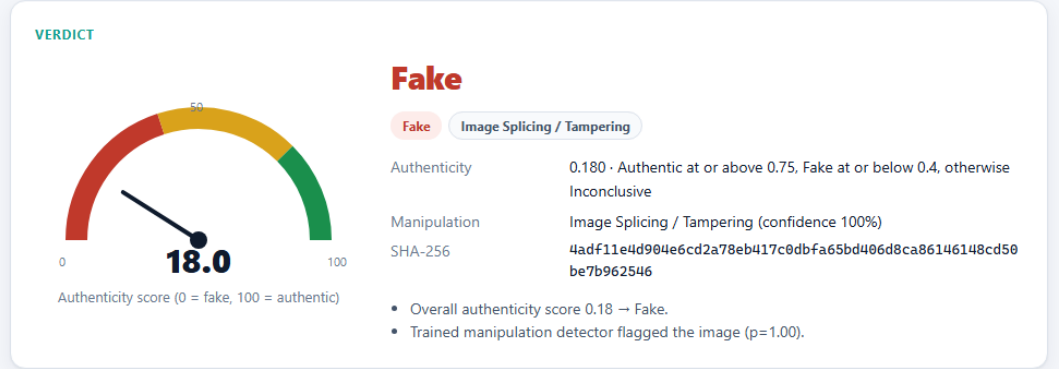


Figure 6.3: Verdict card for a tampered image



Figure 6.4: Agent breakdown for the tampered image: metadata, image forensics with heatmaps, provenance

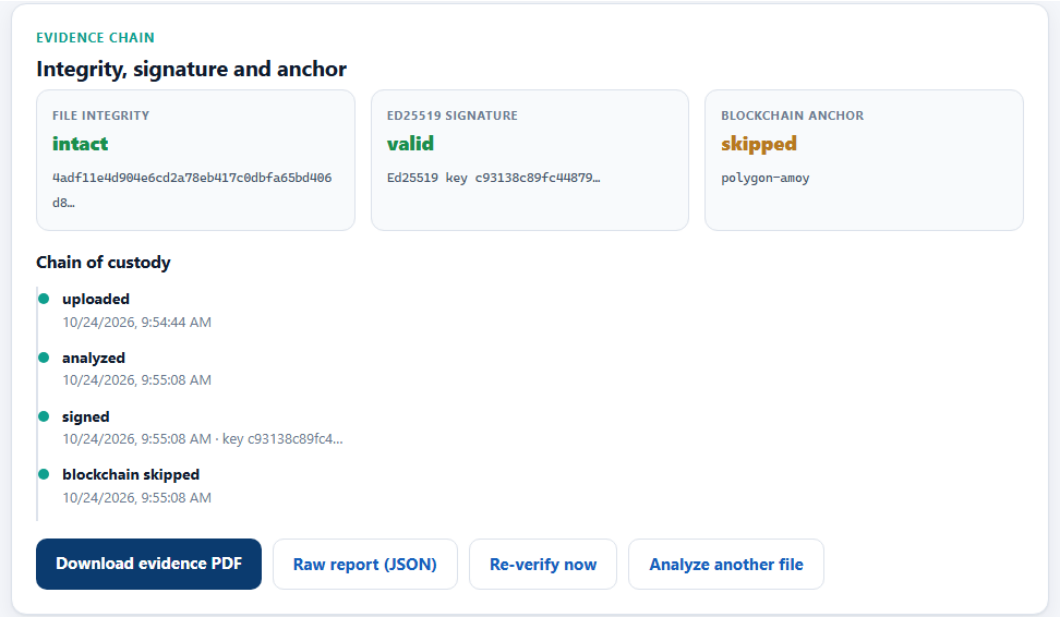


Figure 6.5: Evidence chain: file integrity, Ed25519 signature, anchor status and chain of custody

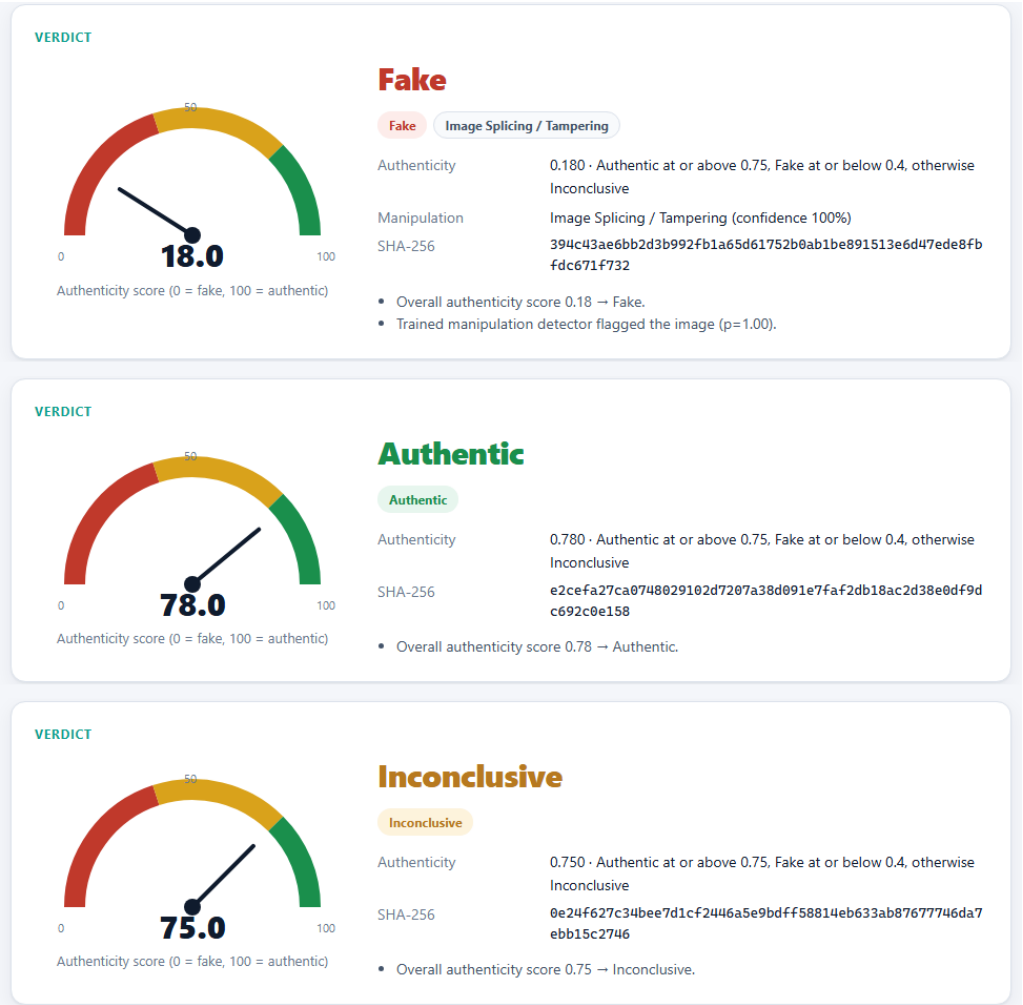


Figure 6.6: Verdict cards of a second tampered image (Fake), an authentic image (Authentic) and an authentic image the engine sends to human review (Inconclusive)

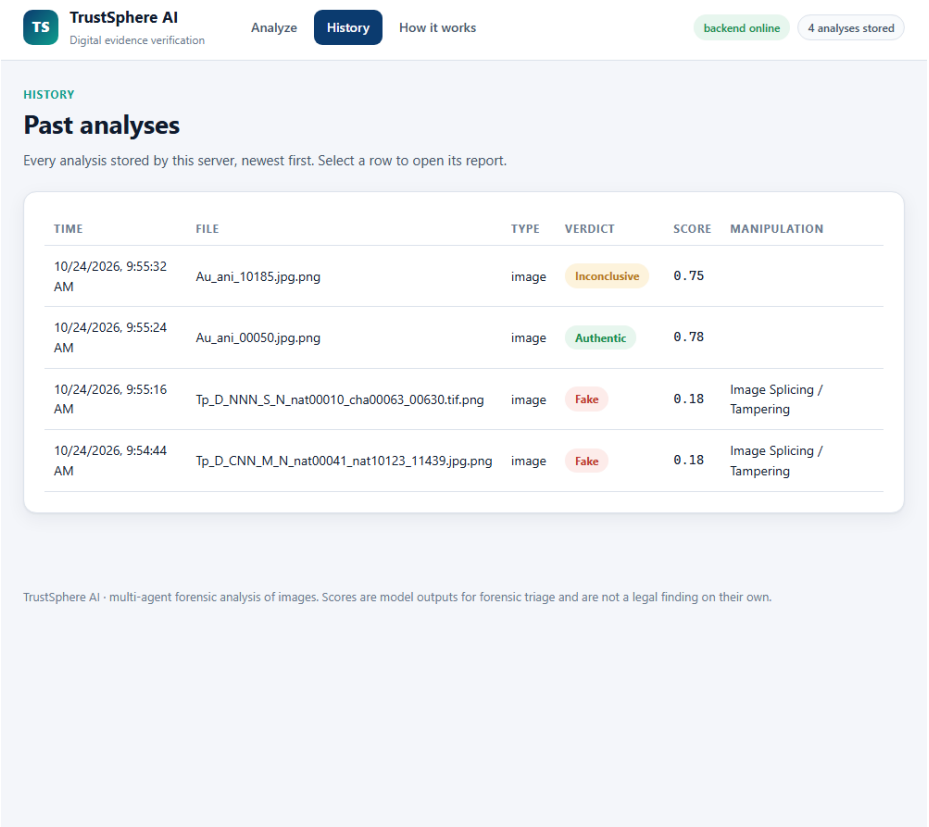


Figure 6.7: History page: every stored analysis with its verdict and score

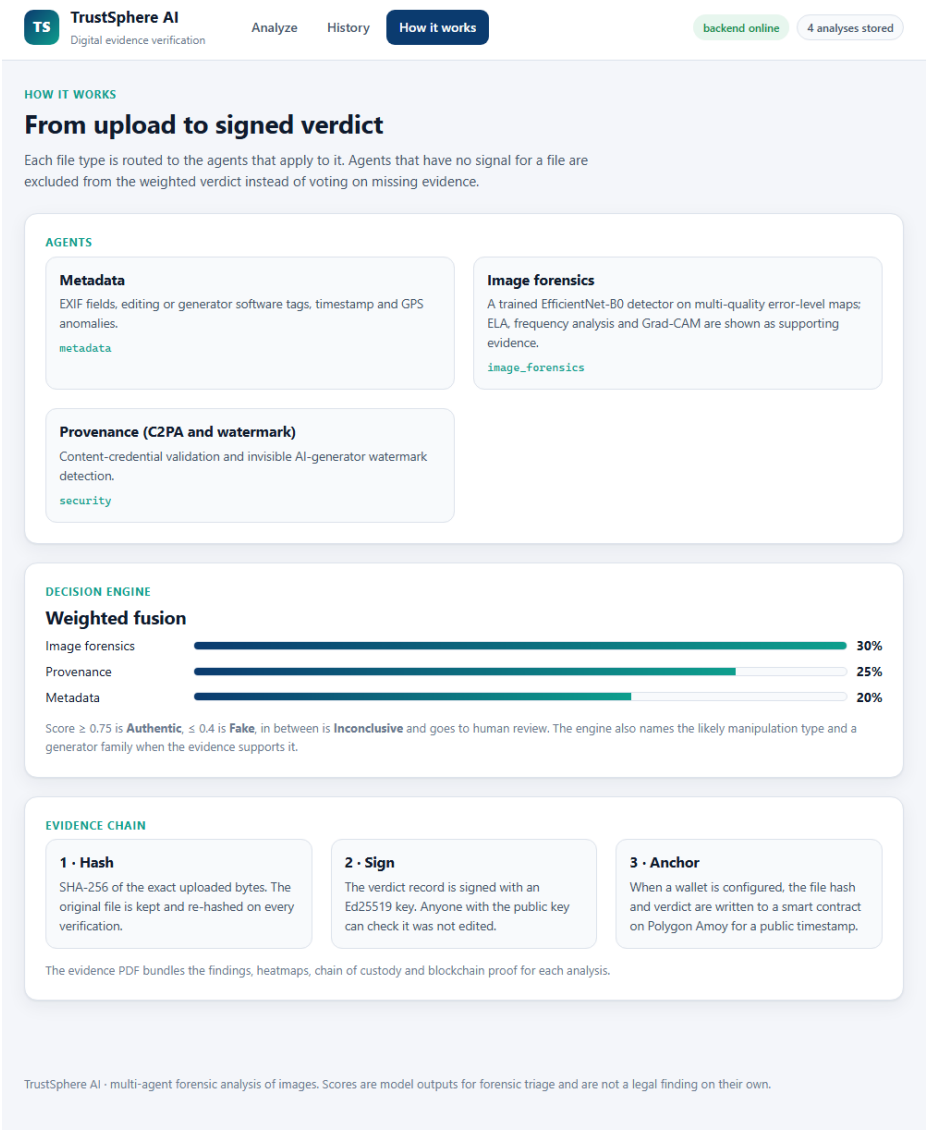
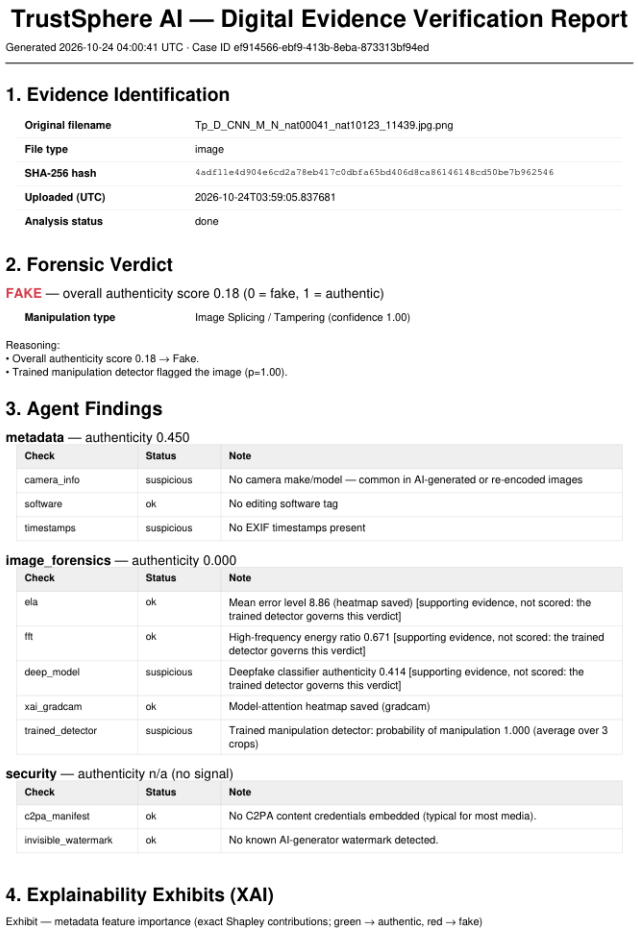


Figure 6.8: How it works page: the agents, the fusion weights and the evidence chain

Each analysis can be downloaded as an evidence PDF containing the identification, verdict, findings, heatmaps, chain of custody and signature.



TrustSphere AI — Digital Evidence Verification Report

Generated 2026-10-24 04:00:41 UTC · Case ID ef914566-ebf9-413b-8eba-873313bf94ed

1. Evidence Identification

Original filename	Tp_D_CNN_M_N_nat00041_nat10123_11439.jpg.png
File type	image
SHA-256 hash	4adf11e4d904e6cd2a78eb417c0dbfa65bd406d8ca86146148cd50be7b962546
Uploaded (UTC)	2026-10-24T03:59:05.837681
Analysis status	done

2. Forensic Verdict

FAKE — overall authenticity score 0.18 (0 = fake, 1 = authentic)

Manipulation type	Image Splicing / Tampering (confidence 1.00)

Reasoning:
• Overall authenticity score 0.18 → Fake.
• Trained manipulation detector flagged the image (p=1.00).

3. Agent Findings

metadata — authenticity 0.450

Check	Status	Note
camera_info	suspicious	No camera make/model — common in AI-generated or re-encoded images
software	ok	No editing software tag
timestamps	suspicious	No EXIF timestamps present

image_forensics — authenticity 0.000

Check	Status	Note
ela	ok	Mean error level 8.86 (heatmap saved) [supporting evidence, not scored: the trained detector governs this verdict]
fft	ok	High-frequency energy ratio 0.671 [supporting evidence, not scored: the trained detector governs this verdict]
deep_model	suspicious	Deepfake classifier authenticity 0.414 [supporting evidence, not scored: the trained detector governs this verdict]
xai_gradcam	ok	Model-attention heatmap saved (gradcam)
trained_detector	suspicious	Trained manipulation detector: probability of manipulation 1.000 (average over 3 crops)

security — authenticity n/a (no signal)

Check	Status	Note
c2pa_manifest	ok	No C2PA content credentials embedded (typical for most media).
invisible_watermark	ok	No known AI-generator watermark detected.

4. Explainability Exhibits (XAI)

Exhibit — metadata feature importance (exact Shapley contributions; green → authentic, red → fake)

Figure 6.9: First page of the evidence PDF of a tampered image

CHAPTER 7

7. RESULT AND DISCUSSION

7.1 Experimental Setup

The datasets are CASIA v2.0 (1,536 authentic and 1,283 tampered images; all PNG in the copy used) and Columbia (183 authentic and 180 spliced images), both from the Hugging Face compilation ductai199x/image-manipulation-dataset-compilation [12]. Label 1 means manipulated. Accuracy, precision, recall, F1, ROC AUC and the equal error rate (EER) are reported; the confusion matrix is shown for each experiment.

7.2 Detector Accuracy on CASIA v2.0

Table 7.1: Detector results on CASIA v2.0

Protocol	Accuracy (%)	Precision	Recall	F1	ROC AUC	EER
Random 5-fold	95.53 ± 0.86	95.16%	95.01%	95.09%	0.9917	4.58%
Source-disjoint 5-fold	95.32 ± 0.80	95.28%	94.39%	94.83%	0.9933	4.47%
Hold-out 20% (588 images)	96.26%	96.49%	95.82%	96.15%	0.9943	3.23%

The two cross-validation protocols give almost the same accuracy (95.53% and 95.32%), so keeping each tampered image with its source did not change the result noticeably. This only controls for the source image named in the dataset metadata, not for every near-duplicate. The fold accuracies were 95.9, 95.4, 96.8, 94.2, 95.4 (random) and 94.0, 96.3, 95.4, 95.9, 95.0 (source-disjoint) percent.

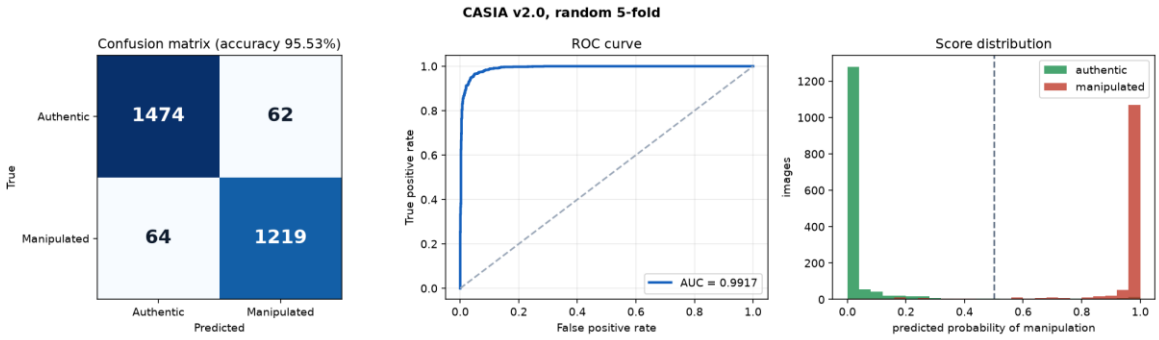


Figure 7.1: CASIA v2.0, random 5-fold cross-validation: confusion matrix, ROC curve and score distribution

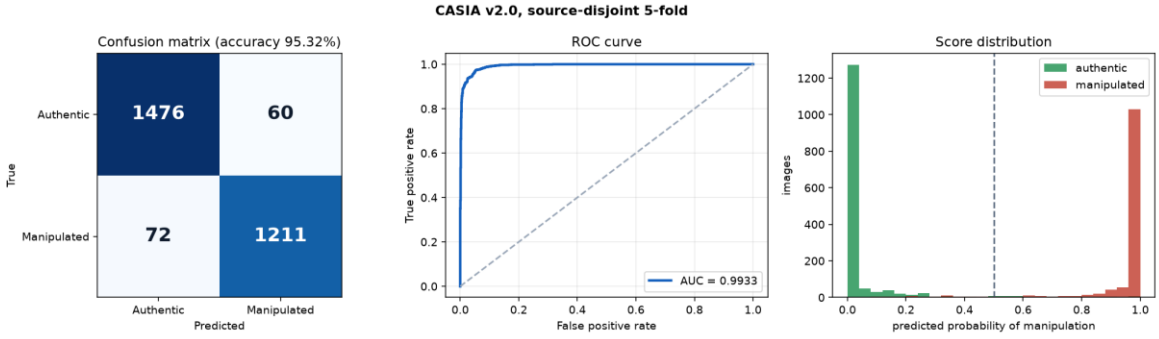


Figure 7.2: CASIA v2.0, source-disjoint 5-fold cross-validation

7.3 The Complete Pipeline on Held-Out Images

The real agents and decision engine were run on the 588 held-out images (301 authentic, 287 tampered). The engine decided 90.5% of them; on those it was right 97.9% of the time. The rest

were sent to review. Counting Inconclusive as a miss, strict accuracy is 88.6%. If the engine were forced to choose by the 0.5 mark of its score it would reach 96.43% (AUC 0.9943).

Table 7.2: Decision-engine verdicts on the held-out images

	Authentic	Fake	Inconclusive
True authentic	251	8	42
True tampered	3	270	14

Authentic images were called Fake in 2.7% of cases and tampered images were called Authentic in 1.0%. Most of the cost of the Inconclusive band falls on authentic images, because the metadata agent gives a PNG with no EXIF about 0.45 and pulls the fused score towards the middle.

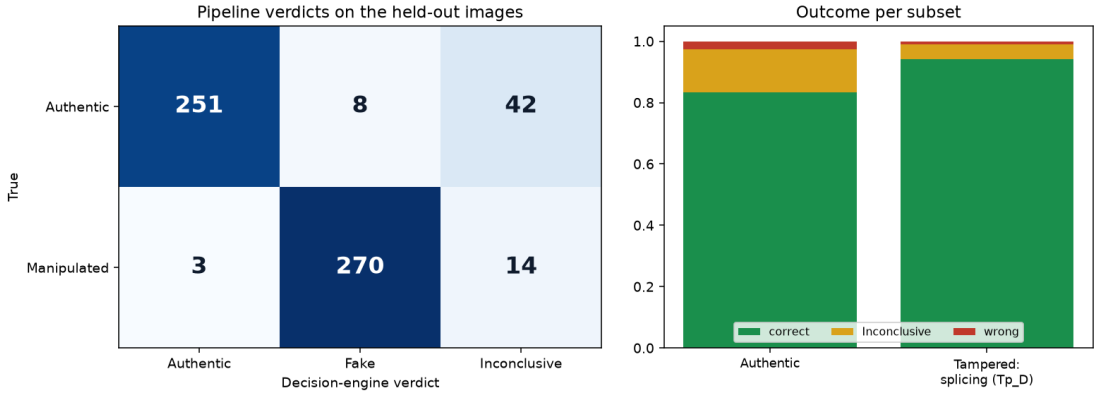


Figure 7.3: Pipeline verdicts on the held-out images

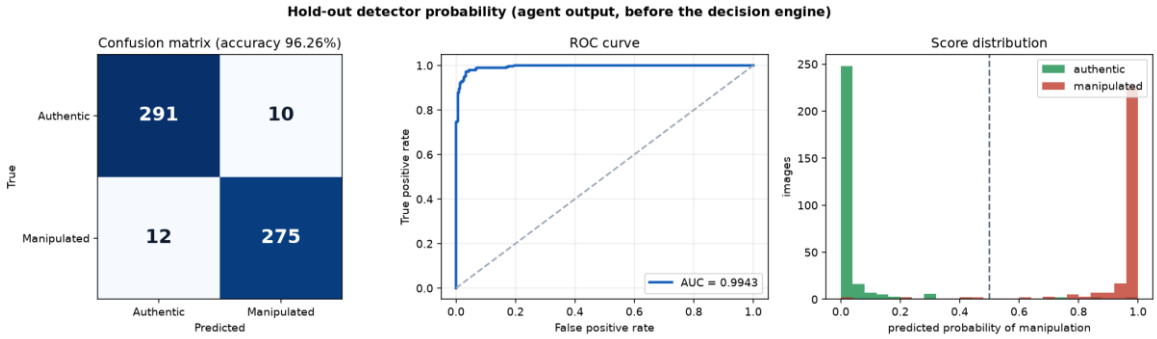


Figure 7.4: Hold-out detector probability before the decision engine

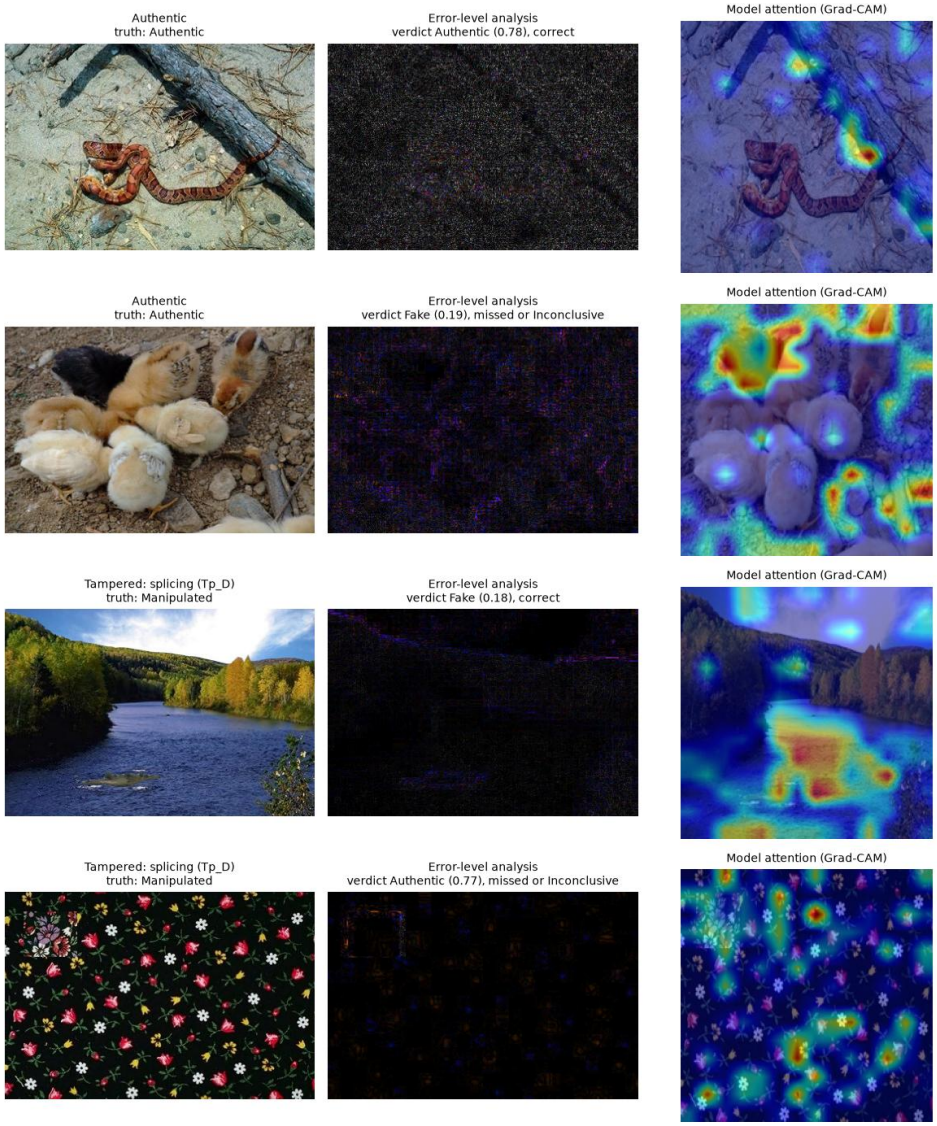


Figure 7.5: Sample outputs from the held-out set: input, error-level analysis and Grad-CAM

7.4 Columbia

Table 7.3: Columbia, 5-fold cross-validation (all variants tried are listed)

Method	Accuracy (%)	Precision	Recall	F1	ROC AUC	EER
Whole image, ELA input	87.60 ± 4.27	84.26%	92.22%	88.06%	0.9610	11.85%
Whole image, SRM noise-residual input	86.49 ± 5.12	81.95%	93.33%	87.27%	0.9490	14.88%
Patch based, SRM input	97.25 ± 2.12	96.70%	97.78%	97.24%	0.9872	2.48%
Patch based, SRM input, scenes kept apart	96.92 ± 2.43	96.69%	97.22%	96.95%	0.9799	2.76%

Columbia consists of uncompressed photographs, so error-level analysis, which relies on a JPEG history, has little to work with, and 363 images are few for training a network on whole images. Three variants were tried, and all are listed above. The whole-image variants are far less stable than on CASIA (their folds differ by more than ten points). The patch-based variant labels 128-pixel patches from the ground-truth masks, scores an image by the mean of its top 5% patch probabilities and takes the decision threshold from the other folds. Because Columbia photographs share scenes, the last row repeats the experiment with every scene kept inside one fold (5 folds);

it is the stricter figure.

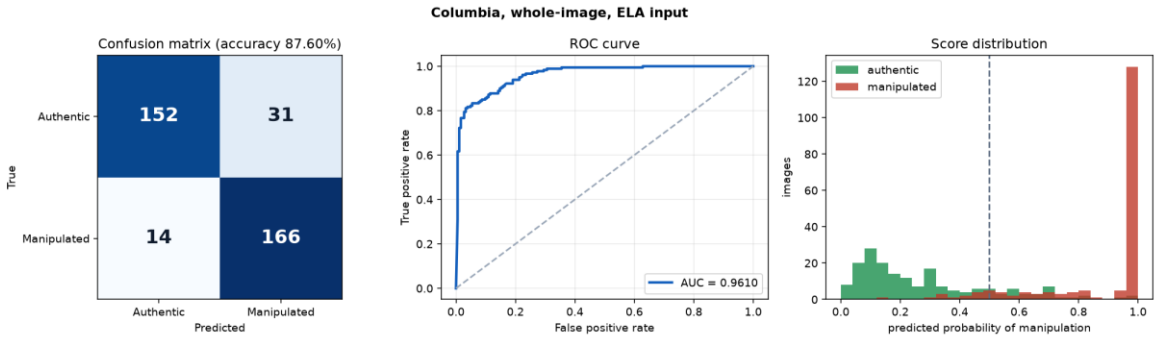


Figure 7.6: Columbia, whole-image detector with ELA input

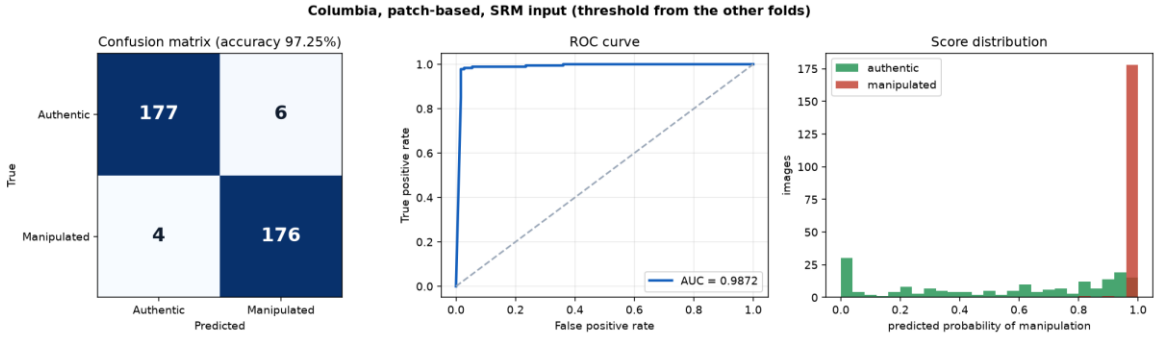


Figure 7.7: Columbia, patch-based detector with SRM input

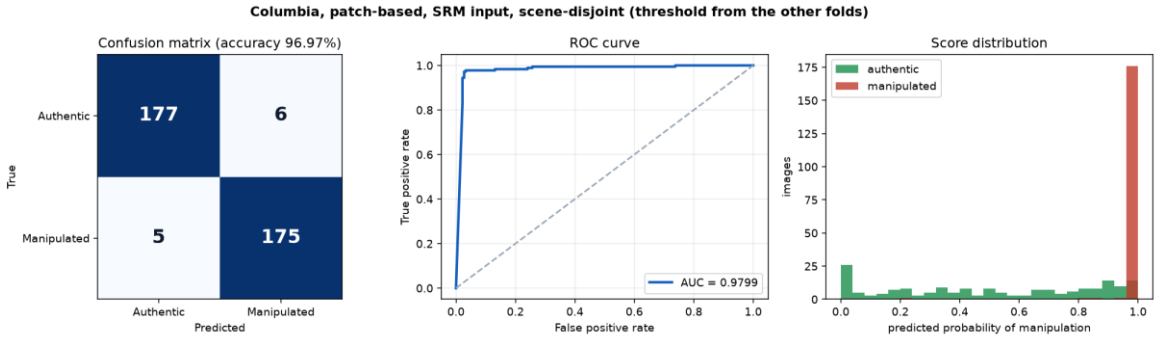


Figure 7.8: Columbia, patch-based detector with scenes kept apart

7.5 Pixel-Level Localisation

Table 7.4: Localiser trained on CASIA v2.0

Test set	Pooled pixel F1	Mean per-image F1	Pixel AUC	Image AUC from the map
CASIA v2.0 validation (source-disjoint)	0.8889	0.7554	0.9569	0.9868
Columbia (never used in training)	0.4330	0.3885	0.6743	0.7994

The localiser finds the tampered region well on images that resemble its training data (pooled F1 0.89) but does not transfer to Columbia (pooled F1 0.43, 0.48 at the best possible threshold). Localisation across datasets is therefore a limitation of this work, and it is not used by the verdict.

7.6 Comparison with the Base Papers

Only the base-paper results that the detector exceeds under a comparable metric on the same dataset are listed. The protocols differ (their splits, subsets and training data), so the table gives context and is not a leaderboard. The full comparison, including results the detector does not reach,

is in the repository file docs/evaluation/EVALUATION.md.

Table 7.5: CASIA v2.0 results of the base papers that the trained detector exceeds

Base paper	Metric	Reported	TrustSphere (random / source-disjoint)
[2] Ravula and Singh, 2025	Accuracy	94.4%	95.53% / 95.32%
[3] Rehman and Jain, 2025	Accuracy	93.57% (mean over runs 93.39%)	95.53% / 95.32%
[4] Deb and Kar, 2026	Precision / recall / F1	87.31 / 90.15 / 88.33	95.16 / 95.01 / 95.09 (random)

Table 7.6: Columbia results of earlier methods listed in [1] that the patch-based detector exceeds

Source	Metric	Reported	TrustSphere (random / scenes kept apart)
Earlier Columbia methods in the review table of [1]	Accuracy	91% to 93.4%	97.25% / 96.97%

7.7 Discussion and Limitations

- The ELA-based detector works well on CASIA v2.0, where most images are JPEG-derived, and poorly on the uncompressed Columbia images. The patch-based detector on noise residuals fixes this on Columbia, but it is a separate model; the deployed application uses the CASIA detector, and a real system will meet both kinds of image.
- Columbia has only 363 images and its photographs share scenes. Keeping each scene in one fold lowers accuracy only slightly, but with so few images the fold-to-fold spread is still several points. The decision threshold of the patch-based detector is set from the other folds' out-of-fold scores; those scores come from models that saw the test fold, so the threshold-free AUC is the safer figure.
- CASIA v2.0 is a benchmark with known quirks: tampered images derive from authentic ones in the same set, so even the source-disjoint split is only partly strict. The held-out test uses the same dataset family and is not evidence about unrelated images.
- The Inconclusive band costs coverage: about one image in ten is sent to review. It is the price of keeping wrong verdicts below about 3% for authentic images and about 1% for tampered images.
- The localiser does not transfer across datasets, so heatmaps are supporting evidence only. The verdict does not depend on them.
- All detector figures are for splicing-style tampering in CASIA and Columbia. Fully AI-generated images, heavy re-compression and screenshots were not evaluated.

7.8 Cost Analysis

Table 7.7: Cost of building and running the system

Item	Cost
Software, models and libraries	Open-source, no licence cost
Datasets	Public, no cost
Database	SQLite locally; free-tier PostgreSQL optional
Blockchain anchoring	Polygon Amoy test network, free test tokens
Training compute	Existing laptop GPU; a cross-validation run takes tens of minutes
Analysis cost per image	About 0.24 s on the GPU (measured on 588 images)

CHAPTER 8

8. CONCLUSION

TrustSphere AI shows that image forensics and evidence integrity can be built as one system. A multi-agent pipeline analyses an uploaded image, a decision engine returns Authentic, Fake or Inconclusive with its reasoning, and every result is hashed, signed, recorded with a chain of custody and exported as an evidence PDF, all behind a web application and a REST API.

The trained detector reaches 95.53% accuracy on CASIA v2.0 by random 5-fold cross-validation and 95.32% when tampered images are kept with their sources, and 96.26% on 588 held-out images; the complete pipeline decides about 90% of them with 97.9% accuracy. These results exceed those reported by the base papers listed in Section 7.6 for CASIA v2.0 under the stated protocols. The work also shows its limits: a whole-image detector is weak on uncompressed Columbia images (a patch-based one reaches about 97%, on a very small dataset), and a localiser trained on CASIA does not transfer to Columbia.

Future work

- Merge the CASIA and Columbia detectors into one model that handles both compressed and uncompressed images, and test it on larger uncompressed sets.
- Evaluate on JPEG-compressed web images and on other benchmarks to measure transfer, and add a calibrated out-of-distribution warning.
- Improve localisation across datasets so that the heatmap can support the verdict.
- Re-balance the metadata agent so that EXIF-free images are not pushed towards Inconclusive, and calibrate the verdict thresholds on held-out data.
- Host the application and add user accounts and access control for case management.

CHAPTER 9

9. REFERENCES

1. R. Ranjan, D. Nandan, M. K. Singh, S. Kumar, K. V. V. Reddy, V. L. Narla and D. K. Jain, "A novel local optimal oriented pattern for image splicing detection leveraging deep learning and SVM," Franklin Open, vol. 16, p. 100634, 2026, doi: 10.1016/j.fraope.2026.100634.
2. J. Ravula and N. Singh, "GAN-ViT-CMFD: A novel framework integrating generative adversarial networks and vision transformers for enhanced copy-move forgery detection and classification with spectral clustering," Intelligent Systems with Applications, vol. 26, p. 200524, 2025, doi: 10.1016/j.iswa.2025.200524.
3. K. Rehman and G. Jain, "Detection of copy-move forgery with deep CNN features using machine learning classifier," Procedia Computer Science, vol. 258, pp. 3062-3071, 2025, doi: 10.1016/j.procs.2025.04.564.
4. P. Deb and N. Kar, "Satyachitra: CNN-based framework for robust copy-move forgery detection and dataset analysis," Procedia Computer Science, vol. 283, pp. 1103-1112, 2026.
5. H.-S. Chen, Y. Zhu, C.-A. Yu, R. Salloum and C.-C. J. Kuo, "GIFT: A green image forgery tracker," Journal of Visual Communication and Image Representation, vol. 118, p. 104797, 2026, doi: 10.1016/j.jvcir.2026.104797.
6. R. Salloum, Y. Ren and C.-C. J. Kuo, "Image splicing localization using a multi-task fully convolutional network (MFCN)," Journal of Visual Communication and Image Representation, vol. 51, pp. 201-209, 2018, doi: 10.1016/j.jvcir.2018.01.010.
7. J. Dong, W. Wang and T. Tan, "CASIA image tampering detection evaluation database," in Proc. IEEE China Summit and International Conference on Signal and Information Processing (ChinaSIP), 2013.
8. Y.-F. Hsu and S.-F. Chang, "Detecting image splicing using geometry invariants and camera characteristics consistency," in Proc. IEEE International Conference on Multimedia and Expo (ICME), 2006.
9. M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in Proc. International Conference on Machine Learning (ICML), 2019.
10. N. Krawetz, "A picture's worth: digital image analysis and forensics," Black Hat Briefings, 2007.
11. R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh and D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in Proc. IEEE International Conference on Computer Vision (ICCV), 2017.
12. ductai199x, "image-manipulation-dataset-compilation," Hugging Face dataset, https://huggingface.co/datasets/ductai199x/image-manipulation-dataset-compilation.
13. D. J. Bernstein, N. Duif, T. Lange, P. Schwabe and B.-Y. Yang, "High-speed high-security signatures," Journal of Cryptographic Engineering, vol. 2, no. 2, pp. 77-89, 2012.
14. Coalition for Content Provenance and Authenticity, "C2PA technical specification," https://c2pa.org.

APPENDIX A – SAMPLE CODE

A.1 Trained detector inference (backend/core/trained_detector.py)

```
def predict(file_path: str) -> dict | None:
    """Return {"p_fake": float, "crops": int, "model": str} or None when no detector
is installed."""
    if not available():
        return None
    import torch
    from PIL import Image, ImageFile
    ImageFile.LOAD_TRUNCATED_IMAGES = True
    img = Image.open(file_path)
    arr = _input_map(img)
    ch = _state["meta"]["channels"]
    mean = np.tile(np.array([0.485, 0.456, 0.406], np.float32), 2)[:ch]
    std = np.tile(np.array([0.229, 0.224, 0.225], np.float32), 2)[:ch]
    crops = _crops(arr)
    x = (np.stack([np.ascontiguousarray(c) for c in crops]).astype(np.float32) /
255.0 - mean) / std
    t = torch.from_numpy(x).permute(0, 3, 1, 2).to(_state["device"])
    with torch.no_grad():
        lg = _state["model"](t).squeeze(1).float()
    return {"p_fake": float(torch.sigmoid(lg.mean())), "crops": len(crops), "model":
_state["meta"].get("name", _state["meta"]["file"])}
```

A.2 Weighted fusion and verdict bands (backend/core/decision_engine.py)

```
AGENT_WEIGHTS = {
    "metadata": 0.20,
    "image_forensics": 0.30,
    "security": 0.25,     # C2PA + watermark provenance (Phase 5)
}

# Verdict thresholds on the authenticity score.
AUTHENTIC_THRESHOLD = 0.75
FAKE_THRESHOLD = 0.40
def aggregate_score(results):
    """Weighted authenticity average across agents.

    Agents may report confidence=None (e.g. security agent when no provenance
    data exists either way) — those are excluded rather than counted as 0.5.
    """
    total_w = 0.0
    acc = 0.0
    for r in results:
        conf = r.get("confidence", 0.5)
        if conf is None:
            continue
        w = AGENT_WEIGHTS.get(r.get("agent"), 0.25)
        acc += float(conf) * w
        total_w += w
    return acc / total_w if total_w > 0 else 0.5



def verdict_from_score(score):
    if score >= AUTHENTIC_THRESHOLD:
        return "Authentic"
    if score <= FAKE_THRESHOLD:
        return "Fake"
    return "Inconclusive"
```

A.3 Training step of the detector (training/train_image_detector.py)

```
def crops_of(arr, k, rng=None, train=False):
    """Crops of size CROP. Train: k random crops. Test: a deterministic grid (at most
k)."""
    h, w, _ = arr.shape
    if h < CROP or w < CROP:
        arr = np.pad(arr, ((0, max(0, CROP - h)), (0, max(0, CROP - w)), (0, 0)),
mode="reflect")
        h, w, _ = arr.shape
```

```
    if train:
        res = []
        for _ in range(k):
            y, x = rng.randint(0, h - CROP), rng.randint(0, w - CROP)
            c = arr[y:y + CROP, x:x + CROP]
            if rng.random() < .5:
                c = c[:, ::-1]
            if rng.random() < .5:
                c = c[::-1]
            res.append(c)
        return res
    ys = np.linspace(0, h - CROP, min(k, max(1, (h - CROP) // 64 + 1))).astype(int)
if h > CROP else [0]
    xs = np.linspace(0, w - CROP, min(k, max(1, (w - CROP) // 64 + 1))).astype(int)
if w > CROP else [0]
    grid = [arr[y:y + CROP, x:x + CROP] for y in ys for x in xs]
    if len(grid) > k:
        idx = np.linspace(0, len(grid) - 1, k).astype(int)
        grid = [grid[i] for i in idx]
    return grid
def train_model(arrs, ys, dev, epochs, repeat, bs, lr, seed, val=None, log=""):
    rng = random.Random(seed)
    torch.manual_seed(seed)
    model = make_model(dev)
    opt = torch.optim.AdamW(model.parameters(), lr=lr, weight_decay=1e-4)
    steps = epochs * ((len(arrs) * repeat + bs - 1) // bs)
    sched = torch.optim.lr_scheduler.OneCycleLR(opt, max_lr=lr, total_steps=steps,
pct_start=0.15)
    scaler = torch.amp.GradScaler("cuda")
    lossf = nn.BCEWithLogitsLoss()
    order = list(range(len(arrs))) * repeat
    for ep in range(epochs):
        model.train()
        rng.shuffle(order)
        tot, n = 0.0, 0
        for b in range(0, len(order), bs):
            idx = order[b:b + bs]
            crops, lab = [], []
            for i in idx:
                crops += crops_of(arrs[i], 1, rng, True)
                lab.append(ys[i])
            x = to_tensor(crops, dev)
            y = torch.tensor(lab, dtype=torch.float32, device=dev)
            with torch.autocast("cuda", dtype=torch.float16):
                loss = lossf(model(x).squeeze(1), y)
            opt.zero_grad(set_to_none=True)
            scaler.scale(loss).backward()
            scaler.step(opt)
            scaler.update()
            sched.step()
            tot += loss.item() * len(idx)
            n += len(idx)
        msg = f"{log} epoch {ep + 1}/{epochs} loss {tot / n:.4f}"
        if val is not None:
            pv = predict(model, val[0], dev)
            msg += f"  val acc {((pv >= .5) == np.array(val[1])).mean():.4f}"
        print(msg, flush=True)
    return model
```